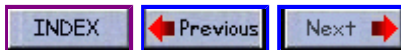


C Programming



Handling User Input In C

[scanf\(\)](#) has problems, in that if a user is expected to type an integer, and types a string instead, often the program bombs. This can be overcome by reading all input as a string (use [getchar\(\)](#)), and then [converting](#) the string to the correct data type.

```
/* example one, to read a word at a time */
#include <stdio.h>
#include <ctype.h>
#define MAXBUFFERSIZE 80

void cleartoendofline( void ); /* ANSI function prototype */

void cleartoendofline( void )
{
    char ch;
    ch = getchar();
    while( ch != '\n' )
        ch = getchar();
}

main()
{
    char    ch; /* handles user input */
    char    buffer[MAXBUFFERSIZE]; /* sufficient to handle one line */
    int     char_count; /* number of characters read for this lin
    int     exit_flag = 0;
    int     valid_choice;

    while( exit_flag == 0 ) {
        printf("Enter a line of text (<80 chars)\n");
        ch = getchar();
        char_count = 0;
        while( (ch != '\n') && (char_count < MAXBUFFERSIZE)) {
            buffer[char_count++] = ch;
            ch = getchar();
        }
        buffer[char_count] = 0x00; /* null terminate buffer */
        printf("\nThe line you entered was:\n");
        printf("%s\n", buffer);

        valid_choice = 0;
        while( valid_choice == 0 ) {
            printf("Continue (Y/N)?\n");
            scanf(" %c", &ch );
            ch = toupper( ch );
            if((ch == 'Y') || (ch == 'N') )
                valid_choice = 1;
            else
                printf("\007Error: Invalid choice\n");
            cleartoendofline();
        }
        if( ch == 'N' ) exit_flag = 1;
    }
}
```

Another Example, read a number as a string

```

/* example two, reading a number as a string */
#include <stdio.h>
#include <ctype.h>
#include <stdlib.h>
#define MAXBUFFERSIZE 80

void cleartoendofline( void );          /* ANSI function prototype */

void cleartoendofline( void )
{
    char ch;
    ch = getchar();
    while( ch != '\n' )
        ch = getchar();
}

main()
{
    char    ch;                          /* handles user input */
    char    buffer[MAXBUFFERSIZE];      /* sufficient to handle one line */
    int     char_count;                  /* number of characters read for this lin
    int     exit_flag = 0, number, valid_choice;

    while( exit_flag == 0 ) {
        valid_choice = 0;
        while( valid_choice == 0 ) {
            printf("Enter a number between 1 and 1000\n");
            ch = getchar();
            char_count = 0;
            while( (ch != '\n') && (char_count < MAXBUFFERSIZE)) {
                buffer[char_count++] = ch;
                ch = getchar();
            }
            buffer[char_count] = 0x00;      /* null terminate buffer
            number = atoi( buffer );
            if( (number < 1) || (number > 1000) )
                printf("\007Error. Number outside range 1-1000\n"
            else
                valid_choice = 1;
        }
        printf("\nThe number you entered was:\n");
        printf("%d\n", number);

        valid_choice = 0;
        while( valid_choice == 0 ) {
            printf("Continue (Y/N)?\n");
            scanf(" %c", &ch );
            ch = toupper( ch );
            if((ch == 'Y') || (ch == 'N') )
                valid_choice = 1;
            else
                printf("\007Error: Invalid choice\n");
            cleartoendofline();
        }
        if( ch == 'N' ) exit_flag = 1;
    }
}

```

Other validation examples

©Copyright B Brown. 1984-1999. All rights reserved.

